

Q.1. Answer the following short questions:

f) drive- ~~n~~dimensional array / having rowez bound zero)

1qbr-ovder

$[u_1][u_2][u_3]$

toy

ten q

2/0/2

address 100. The dimensions u_1, u_2, u_3 are 10, 5, and 3. you're required to calculate address of location

e

te

memo

Address of $A[i][j][k] = \text{Base} + \text{size}((i - LB_i) \times M \times N$

$+ (j - LB_j) \times N + (k - LB_k))$

Base = 100, size = 4, M = 5, N = 3

$LB_i = LB_j = LB_k = 0, i = 6, j = 3, k = 1$

$$\begin{aligned}\text{Address of } A[i][j][k] &= 100 + 4((6)5 \times 3 + (3) \times 3 + (1)) \\ &= 100 + 4(90 + 9 + 1) \\ &= 100 + 4(100) \\ &= 500\end{aligned}$$



ii) Obtain step count of following code segment.

for($i=0$; $i < n \times n \times n$; $i++$)

$\downarrow$ temp; $\downarrow$ $\downarrow$
 $\downarrow$ $\downarrow$ n^3+1 n^3
 1 n^3

$$\text{stepCount} = 1 + n^3 + 1 + n^3 + n^3$$

$$= 3n^3 + 2$$

$$O(f(n)) = O(n^3)$$

iii) The running time of a program is $N^3 + 2N^2 + 3N + 4$, where N is input size. Find tight bound for growth rate of program. Do not forget to mention the constants (c and N_0).

$$f(n) = N^3 + 2N^2 + 3N + 4$$

bounds:- $g(n) = N^3$

$$O(g(n)) = O(N^3)$$

$$\Theta(g(n)) = \Theta(N^3)$$

$$\Omega(g(n)) = \Omega(N^3)$$

value of c and N_0 :-

$$f(n) \leq c \cdot g(n)$$

$$N^3 + 2N^2 + 3N + 4 \leq c \cdot N^3$$

$$1 + \frac{2}{N} + \frac{3}{N^2} + \frac{4}{N^3} \leq c$$

$N=1$ $1 + 2 + 3 + 4 \leq c$

$$10 \leq c$$

$N=2$ $1 + 1 + 0.75 + 0.5 \leq c$

$$3.25 \leq c$$

$N=3$ $2.148 \leq c$

$N=4$ $1.75 \leq c$

N_0	c
1	10
2	3.25
3	2.148
4	1.75
1	1
∞	1

4) An algorithm takes 12 sec for input size 12000. How large a problem can be solved in one minute if running time is $10N$?

$$T(N) = 10N \rightarrow \Theta(N)$$

know

$$T(12,000) = 12 \text{ sec.}$$

$$\text{but } T(12,000) = 10 \times 12,000 \\ = 120,000$$

There must be some constant k .

$$k = \frac{12 \text{ (Actual Time)}}{120,000 \text{ (Hypothetical)}} = \frac{1}{10,000}$$

So,

$$T(N) = k \times 10N$$

publend IIs to sa/yeo/

one-minute/60-sec

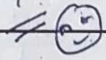
$$60 = k \times 10N$$

$$60 = \frac{1}{10,000} \times 10N$$

$$60,000 = N$$

$$1 \text{ sec} = 1,000 \text{ milliseconds.}$$

$$60 \text{ sec} = 60,000 \text{ milliseconds.}$$



5) convert $A * ((B+C) - D)$ into its equivalent postfix form.

$$A * ((B+C) - D)$$

input	expression	stack
A	A	#
*	A	*
(	A	*(
(	A	*((
B	AB	*((
+	AB	*((+
)	AB+	*(
-	AB+	*(-
D	AB+D	*(-
)	AB+D-	*
	AB+D-*	

PostFix: $AB+D-*$

Q.2. Answer the following questions:

i) sort the following array into increasing order using insertion sort. Clearly show the contents of array after each iteration of outer loop.

	13, 15, 17, 12, 19, 18, 10, 16, 11	
Pass 1:	13, 15, 17, 12, 19, 18, 10, 16, 11	key = 15
Pass 2:	13, 15, 17, 12, 19, 18, 10, 16, 11	key = 17
Pass 3:	12, 13, 15, 17, 19, 18, 10, 16, 11	key = 12
Pass 4:	12, 13, 15, 17, 19, 18, 10, 16, 11	key = 19
Pass 5:	12, 13, 15, 17, 18, 19, 10, 16, 11	key = 18
Pass 6:	10, 12, 13, 15, 17, 18, 19, 16, 11	key = 10
Pass 7:	10, 12, 13, 15, 16, 17, 18, 19, 11	key = 16
Pass 8:	10, 11, 12, 13, 15, 16, 17, 18, 19	key = 11
sorted array: 10, 11, 12, 13, 15, 16, 17, 18, 19		

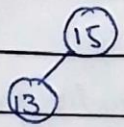
(2) Draw Binary Search Tree (BST) when the following values are inserted ~~by~~ one-by-one.

15, 13, 18, 17, 14, 12, 20, 10, 19, 11

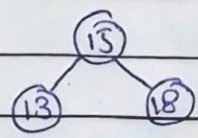
(1) 15



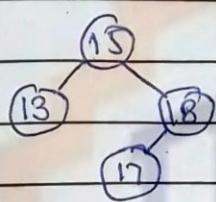
(2) 13



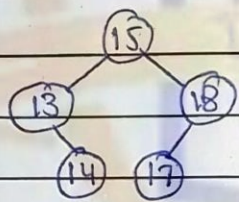
(3) 18



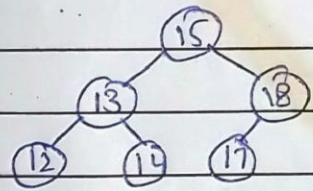
(4) 19



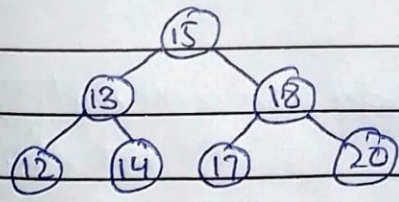
(5) 14



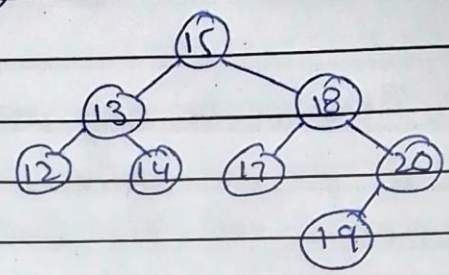
(6) 12



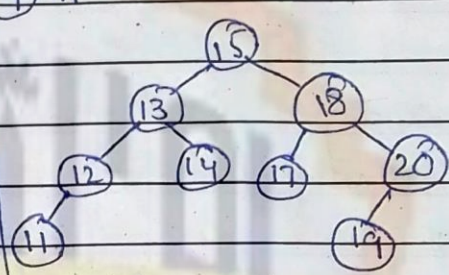
(7) 20



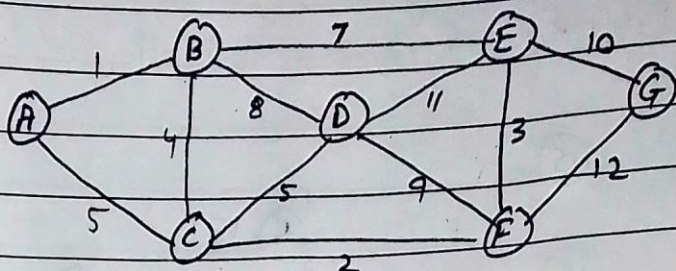
(8) 19



(9) 11



③ Given the following un-directed weighted graph G :



There can be many possible weighted acyclic graph exist in G , you are required to find and draw the one, connected with the least total weight.

There are two methods to make such graph:

- i) Prim's Method ii) Kruskal's Method.

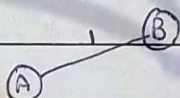
⇒ using Prim's Method.

⇒ starting from (A).

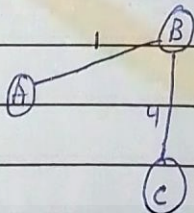
1

(A)

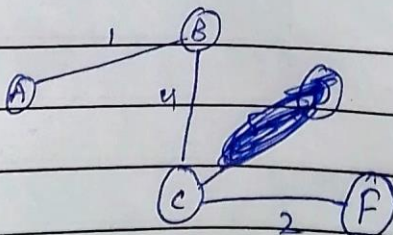
2 Connect (A) to least weight edge



3 Connect next least weight edge.



4 Repeat step 3 until all nodes are connected.

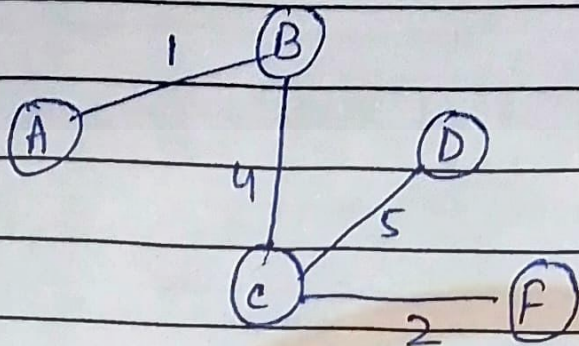


(A) — (C) Not safe.

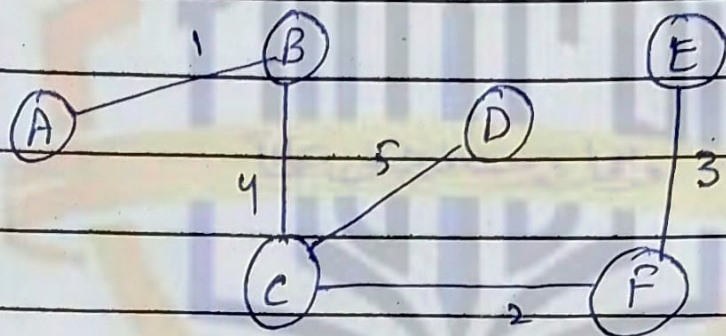
(C) — (D) safe

(C) — (F) safe

6



7



8

